

StellA: Subspace Learning in Low-rank Adaptation using Stiefel Manifold

Zhizhong Li, Sina Sajadmanesh, Jingtao Li, Lingjuan Lyu*

Sony AI

Zurich, Switzerland

{zhizhong.li,sina.sajadmanesh,jingtao.li,lingjuan.lv}@sony.com

Abstract

Low-rank adaptation (LoRA) has been widely adopted as a parameter-efficient technique for fine-tuning large-scale pre-trained models. However, it still lags behind full fine-tuning in performance, partly due to its insufficient exploitation of the geometric structure underlying low-rank manifolds. In this paper, we propose a geometry-aware extension of LoRA that uses a three-factor decomposition $USV^\top$. Analogous to the structure of singular value decomposition (SVD), it separates the adapter’s input and output subspaces, V and U , from the scaling factor S . Our method constrains U and V to lie on the Stiefel manifold, ensuring their orthonormality throughout the training. To optimize on the Stiefel manifold, we employ a flexible and modular geometric optimization design that converts any Euclidean optimizer to a Riemannian one. It enables efficient subspace learning while remaining compatible with existing fine-tuning pipelines. Empirical results across a wide range of downstream tasks, including commonsense reasoning, math and code generation, image classification, and image generation, demonstrate the superior performance of our approach against the recent state-of-the-art variants of LoRA. Code is available at <https://github.com/SonyResearch/stella>.

1 Introduction

The rise of large foundation models [2, 20] has transformed machine learning, driving breakthroughs across a diverse range of applications [54, 60, 36]. These models, often with billions of parameters, show exceptional performance in fields such as natural language understanding [21], computer vision [70], and multi-modal learning [62]. Yet, their substantial scale results in massive computational and storage costs, limiting broader adoption via task-specific fine-tuning [27].

To address this challenge, parameter-efficient fine-tuning (PEFT) methods, such as prefix tuning [32], prompt tuning [45], and adapter tuning [27], have gained considerable attention. Among these, Low-Rank Adaptation (LoRA) [27] is widely adopted due to its ability to efficiently adapt pre-trained models to downstream tasks without altering the original network architecture or incurring extra inference costs. LoRA operates by learning low-rank adaptations to a selected set of linear layers while keeping the original weights frozen. Given a pre-trained weight matrix $W \in \mathbb{R}^{m \times n}$, LoRA computes the adapted weight as $W + BA^\top$, where $B \in \mathbb{R}^{m \times r}$ and $A \in \mathbb{R}^{n \times r}$. Choosing a small $r \ll \min(m, n)$ significantly reduces the number of trainable parameters. Nevertheless, a performance gap often exists between LoRA and full fine-tuning due to the limited capacity of the low-rank matrices A and B to capture complex updates required for optimal task performance [22].

A common strategy to improve LoRA’s performance is to guide its initialization by leveraging structural insights from the pre-trained model [67]. Particularly, singular value decomposition

*Corresponding author

(SVD) has been used for uncovering informative subspaces in weight matrices, activations, or gradients [38, 59, 41]. SVD factorizes a given matrix M into $M = U\Sigma V^\top$, where the columns of U and V form orthonormal bases for the output and input spaces of M , respectively, ordered by the importance dictated by the singular values in Σ . This property makes SVD well-suited for identifying low-dimensional subspaces that preserve meaningful information from the pre-trained model.

While SVD-based initialization methods for LoRA have shown promise, their influence is limited to the start of training, offering minimal guidance for the subsequent optimization process. This naturally leads to the question: *whether explicitly optimizing the subspace throughout training can yield further performance improvements?* Moreover, the existence of various heuristics for subspace selection—such as focusing on leading vs. trailing components [38, 58], or considering weights vs. gradients [38, 59]—suggests that intuition-driven manual subspace selection is suboptimal. This motivates a more principled approach to learn subspaces from data throughout training.

To bridge this gap, we propose a novel approach that directly optimizes the input-output subspaces of LoRA during training. Our key insight is to mirror the structure of the truncated SVD by representing the low-rank adaptation for weight matrix W as a three-factor formulation,

$$W + USV^\top, \quad (1)$$

where $U \in \mathbb{R}^{m \times r}$ and $V \in \mathbb{R}^{n \times r}$ define the orthonormal bases for the output and input subspaces, respectively, and $S \in \mathbb{R}^{r \times r}$ captures the transformation between them. To ensure that the subspace parameters U and V remain orthonormal during training, we constrain them to lie on the *Stiefel manifold*—the set of matrices with orthonormal columns. This leads to our method, **Stiefel Low-rank Adaptation (Stella)**, which performs LoRA using a subspace-aware formulation. By leveraging the geometric optimization on the Stiefel manifold, Stella maintains the geometric structure of the subspaces in training, allowing for principled and effective learning of low-rank adaptation.

Recently, methods such as DoRA [35] found that decomposing the weight into magnitude and direction components can improve LoRA’s performance. This aligns with our approach, as the Stiefel manifold constraint on U and V captures the direction, and S models the magnitude. Besides, maintaining the orthogonality of low-rank matrices U and V during training is also beneficial for certain downstream analyses, such as adversarial robustness [50, 52].

We benchmark Stella across a diverse set of domains, including natural language understanding, natural language generation, visual understanding, and visual generation. Compared with the state-of-the-art LoRA variants, Stella consistently achieves superior performance across all evaluated tasks. Relative to the strongest baseline, Stella achieves notable improvements: up to +1.3 accuracy points on commonsense reasoning, +2.33 on math and code generation, +0.25 on image classification, and a 7.11 point reduction in FID for text-to-image generation.

Our contributions are summarized as follows: (1) We propose a novel three-factor representation for LoRA incorporating Stiefel manifold constraints, enabling the optimization of LoRA’s input and output subspaces directly during training. (2) We present a flexible geometric optimization algorithm for the Stiefel manifold, allowing seamless integration with existing Euclidean optimizers. (3) We conduct ablation studies on the design choices and evaluate the impact of different geometric structures. (4) We verify the effectiveness of Stella across a variety of tasks and models, encompassing natural language understanding and generation, image classification, and text-to-image generation.

2 Related Work

LoRA Initialization Methods and the Use of SVD. Several recent works explore improved LoRA initialization using matrix decomposition techniques like SVD to better align the adaptation subspace with pretrained weights. PiSSA [38] and LaMDA [4] initialize adapters using the leading r singular vectors from SVD of the pretrained weights, while MiLoRA [58] uses the trailing r singular vectors. EVA [41] applies SVD to activations and adapts rank based on the spectrum, while LoRA-GA [59] uses SVD on gradients to align with task-relevant directions. Beyond SVD, OLoRA [10] employs QR decomposition and selects the first r columns from the orthonormal matrix. These approaches exploit meaningful subspaces to improve initialization, whereas ours directly learns the optimal subspace during training, allowing for more flexible and effective task-specific adaptation.

Geometric Constraints for LoRA. Zhang and Pilanci [66] use Riemannian geometry to precondition the LoRA optimization, whereas we optimize the adapter’s subspaces using the Stiefel manifold, with a three-factor decomposition. Other methods impose orthogonality constraints on LoRA weights. OFT [46] learns orthonormal rotations of pretrained weights via Cayley parameterization. Spectral Adapter [67] fine-tunes within the top spectral subspace, with a variant that rotates leading singular vectors using orthonormal matrices. In contrast, our approach enforces orthogonality directly on the input and output subspace projections via Stiefel manifold optimization, offering a more flexible and expressive adaptation mechanism. Concurrently, PoLAR [34] also sets U and V on Stiefel manifold, but it uses a landing algorithm with infeasibility penalty instead of retraction for optimization.

LoRA with Three Factors. TriLoRA [19] and MoSLoRA [61] use the three-factor formulation for LoRA, but they do not keep the orthogonality of the two projections during training. LoRA-XS [5] also uses the three-factor formulation, but they freeze the two projections and only train the middle $r \times r$ matrix for extreme efficiency. Both AdaLoRA [68] and GeoLoRA [53] adopt a three-factor formulation, with U and V constrained to be orthogonal matrices. AdaLoRA achieves orthogonality via regularization, while GeoLoRA uses gradient flow. However, their focus is to achieve rank adaptability by inspecting the singular values of S . StelLA can be readily combined with AdaLoRA’s rank adaptation strategy by constraining S to be diagonal, which we leave to future work.

Other LoRA Variants. Various extensions have been proposed to improve LoRA’s efficiency and performance. rsLoRA [28] introduces a scaling factor based on the square root of the rank for better stability. LoRA+ [23] accelerates convergence by applying separate learning rates to the two matrices. DoRA [35] and DeLoRA [7] decouple LoRA updates into magnitude and direction. ReLoRA [33] periodically merges adapter weights to improve expressiveness. QLoRA [15] integrates quantization with LoRA to minimize memory usage. These efforts are orthogonal to our work.

3 Preliminaries

Here we briefly review the related concepts in differential geometry and geometric optimization. For a more detailed introduction, we refer the reader to Edelman et al. [17], Absil et al. [1], Roy et al. [49], Becigneul and Ganea [6]. We also prepared an intuitive example in low dimensions in Appendix A to help readers understand the geometric concepts. The Stiefel manifold, denoted $\text{St}(k, n)$, is the set of all $n \times k$ matrices with orthonormal columns, *i.e.*, $\text{St}(k, n) = \{Y \in \mathbb{R}^{n \times k} \mid Y^\top Y = I_k\}$. To optimize a function $f : \text{St}(k, n) \rightarrow \mathbb{R}$, we require tools from the Riemannian geometry. The tangent space at a point $Y \in \text{St}(k, n)$, denoted $T_Y \text{St}(k, n)$, consists of matrices $\Delta \in \mathbb{R}^{n \times k}$ satisfying $Y^\top \Delta + \Delta^\top Y = 0$. A Riemannian metric defines an inner product on the tangent space, and the canonical metric in Stiefel manifold is $g_Y(\Delta_1, \Delta_2) = \text{tr}(\Delta_1^\top (I_n - \frac{1}{2}YY^\top)\Delta_2)$. The Riemannian gradient grad_Y w.r.t. function f can be computed from the Euclidean gradient ∇_Y by

$$\text{grad}_Y = \nabla_Y - Y \nabla_Y^\top Y. \quad (2)$$

If the Riemannian gradient is modified, *e.g.*, by adding momentum, it may no longer lie in the tangent space of the manifold. To perform geometric optimization, we need to project it back to the tangent space using the projection $\pi_Y : T_Y \mathbb{R}^{n \times k} \rightarrow T_Y \text{St}(k, n)$,

$$\pi_Y(\Delta) = \Delta - Y \text{symm}(Y^\top \Delta), \quad (3)$$

where $\text{symm}(A) = \frac{1}{2}(A + A^\top)$ symmetrizes a matrix. After taking a step along a tangent vector Δ , the resulting point $Y + \Delta$ typically leaves the manifold. Hence, a retraction is used to map it back. In this work, we use the polar retraction, defined as

$$\rho_Y(\Delta) = \text{uf}(Y + \Delta), \quad (4)$$

where $\text{uf}(\cdot)$ returns the orthogonal matrix in the polar decomposition.

4 Subspace Learning in LoRA Using Stiefel Manifold

We now present Stiefel Low-rank Adaptation (StelLA), which learns the input and output subspaces of the adapter directly during fine-tuning. Given a pre-trained weight matrix $W \in \mathbb{R}^{m \times n}$, the goal is to fit it to a downstream task using a three-factor low-rank adapter,

$$\tilde{W} = W + \frac{\alpha}{r} USV^\top, \quad (5)$$

Algorithm 1 StelLA: Stiefel Low-Rank Adaptation

Require: Pre-trained weight $W \in \mathbb{R}^{m \times n}$, loss function $\mathcal{L}$, a Euclidean optimizer’s step function ‘step’, rank r , scale factor α , number of iterations T .

- 1: Randomly initialize $U_0 \in \text{St}(r, m)$ and $V_0 \in \text{St}(r, n)$, set $S_0 \leftarrow I_r$.
 - 2: **for** $t \leftarrow 0$ to $T - 1$ **do**
 - 3: Compute loss: $\mathcal{L}_t \leftarrow \mathcal{L}(W + \frac{\alpha}{r} U_t S_t V_t^\top)$.
 - 4: Compute Euclidean gradients: $\nabla_{U_t}, \nabla_{S_t}, \nabla_{V_t}$. ▷ via automatic differentiation
 - 5: Convert Euclidean gradients to Riemannian: ▷ Equation (2)
$$\text{grad}_{U_t} \leftarrow \nabla_{U_t} - U_t \nabla_{U_t}^\top U_t, \quad \text{grad}_{V_t} \leftarrow \nabla_{V_t} - V_t \nabla_{V_t}^\top V_t.$$
 - 6: Take tentative steps using the given optimizer’s step function: ▷ e.g., using Adam
$$\tilde{U}_{t+1} \leftarrow \text{step}(U_t, \text{grad}_{U_t}), \quad \tilde{V}_{t+1} \leftarrow \text{step}(V_t, \text{grad}_{V_t}), \quad S_{t+1} \leftarrow \text{step}(S_t, \nabla_{S_t}).$$
 - 7: Project the perturbed gradients $\tilde{U}_{t+1} - U_t, \tilde{V}_{t+1} - V_t$ back to the tangent space: ▷ Equation (3)
$$\Delta_{U_t} \leftarrow \pi_{U_t}(\tilde{U}_{t+1} - U_t), \quad \Delta_{V_t} \leftarrow \pi_{V_t}(\tilde{V}_{t+1} - V_t).$$
 - 8: Update and retract back to the manifold: $U_{t+1} \leftarrow \rho_{U_t}(\Delta_{U_t}), V_{t+1} \leftarrow \rho_{V_t}(\Delta_{V_t})$. ▷ Equation (4)
 - 9: **end for**
 - 10: **return** Adapted weight: $\tilde{W} \leftarrow W + \frac{\alpha}{r} U_T S_T V_T^\top$.
-

where $U \in \text{St}(r, m)$ provides an orthonormal basis for the output subspace, $V \in \text{St}(r, n)$ provides an orthonormal basis for the input subspace, and $S \in \mathbb{R}^{r \times r}$ learns a mapping. $\alpha \in \mathbb{R}$ is a scale hyperparameter. The rank r is often much smaller than the dimensions of W , i.e., $r \ll \min(m, n)$.

Let $\mathcal{L}$ denote the task-specific loss. We optimize the following objective (for notational brevity, we express the objective for one StelLA layer):

$$\min_{U \in \text{St}(r, m), S \in \mathbb{R}^{r \times r}, V \in \text{St}(r, n)} \mathcal{L} \left(W + \frac{\alpha}{r} U S V^\top \right), \quad (6)$$

leading to a constrained optimization over Stiefel manifolds for U and V , and an unconstrained optimization for S .

The optimization is carried out using Algorithm 1. The algorithm begins by initializing U_0 , V_0 , and S_0 (line 1). At training iteration t , the loss is computed based on the adapted weight in the forward pass (line 3), and Euclidean gradients with respect to U_t , S_t , and V_t are obtained (line 4) in the backward pass using standard automatic differentiation in deep learning frameworks. Since U_t and V_t must remain on the Stiefel manifold, their gradients are converted to Riemannian gradients using Equation (2) (line 5). These gradients, along with the gradient of S_t , are passed to an existing Euclidean optimizer’s step function, e.g., using SGD, Adam, or RMSProp, to generate tentative updates (line 6). The perturbed gradient by the optimizer is reverse-engineered by examining the difference $\tilde{U}_{t+1} - U_t$. Since the perturbed gradient may not lie on the tangent space of the manifold, it is projected back to the tangent spaces at the current points (line 7). Then, the update is performed by using the polar retraction (line 8). The algorithm ensures that U and V evolve in the Stiefel manifold throughout training. After T steps, the final adapted weight is returned (line 10).

Initialization. Both U and V are initialized with random column-orthonormal matrices [51], and we initialize S with the identity matrix. In prior work [38], when the adapter is initialized with non-zero values, the initialization is subtracted from the pre-trained weight matrix to simulate a zero-initialized adapter. We empirically find this trick to be unnecessary in our case. An in-depth ablation of initialization strategies is studied in Section 5.5.

Implementation. Algorithm 1 is designed to be modular and flexible, enabling integration with any existing Euclidean optimizer. The ‘step’ function abstracts the optimizer’s internal logic, such as momentum updates or adaptive learning rates, allowing StelLA to seamlessly integrate with a wide range of optimizers, including SGD, Adam, and others, while cleanly separating geometric constraints from the choice of optimization algorithm. We implement StelLA in PyTorch [43] using optimizer hooks. Specifically, line 5 is implemented as a pre-hook to the optimizer step, while lines 7–8 are implemented as a post-hook. Our implementation is readily integrable with HuggingFace’s PEFT library [37], enabling easy adoption by the community.

The polar retraction is computed via SVD, which is the most expensive operation in the algorithm. To improve efficiency, we stack all U s and V s with identical shapes across different layers and apply a batched SVD. This batched strategy yields 15-20 $\times$ speed up in our experiments, effectively eliminating the computational bottleneck when scaling to large models with many adapted layers. Please refer to Appendix D for a detailed discussion.

Complexity. StelLA adds $r(m + n) + r^2$ parameters per adapted layer, which is r^2 parameters more than LoRA. This additional memory footprint is negligible since $r \ll \min(m, n)$. In our experiments, we show that the number of parameters in StelLA is comparable to LoRA and DoRA [35]. Nevertheless, in Section 5.5, we show that the superior performance of StelLA is not merely the result of this tiny increase in the number of trainable parameters but our geometric formulation by relaxing the orthonormality constraints. Regarding inference, similar to LoRA, we can merge the adapted weights into the original ones, resulting in a single weight matrix with no overhead.

Gradient Scaling. Previous work such as LoRA+ [23] showed that setting different learning rates for B and A in LoRA can improve convergence speed. The core idea is to ensure that both B and A are efficiently updated during training. Motivated by their insights, we balance the learning speed of U and V in StelLA via gradient scaling. For a random unit vector $x \in \mathbb{R}^m$, it is easy to show that the variance of each element is $\frac{1}{m}$. Since the columns of U and V are unit vectors, their individual entries are expected to have magnitudes on the order of $1/\sqrt{m}$ and $1/\sqrt{n}$, respectively. Adam-style optimizers, however, normalize gradients so that their coordinate-wise variance is $\Theta(1)$ [64]. This means that the learning speed—the ratio between the average magnitude of the effective gradient and the average magnitude of the parameter—for the entries of U and V are different when $m \neq n$. This imbalance is particularly pronounced in feed-forward layers of LLMs, where the hidden dimension is typically enlarged and then shrunk by a factor of 4 [9], causing the taller matrix to learn two times faster. To compensate for this difference, before applying the projection operation (line 7 in Algorithm 1), we scale the gradients of U and V by $\sqrt{d/m}$ and $\sqrt{d/n}$, respectively, where d is a hyperparameter. In our experiments, we set d equal to the hidden dimension of the input tokens.

Comparison to Existing Geometric Optimizers. Existing geometric optimization methods, such as Riemannian SGD [49] and Riemannian Adam [6], can handle optimization over generic manifolds. However, these methods are intrinsically coupled to specific optimizers, as they rely on accessing and manipulating internal states such as momentum or adaptive learning rates. For instance, Riemannian Adam parallel transports the momentum vector across tangent spaces to maintain consistency in updates. This tight integration limits their generalizability to other optimization algorithms. In contrast, our approach decouples geometric constraints from the choice of base Euclidean optimizer by treating its update direction as a perturbation to the Riemannian gradient. The projection operator π (line 7 in Algorithm 1) ensures that the perturbed gradient is mapped back to the tangent space of the current point. This allows us to use any Euclidean optimizer without modifying its internal logic.

5 Experiments

We conduct extensive experiments to evaluate the performance of StelLA on various domains: natural language understanding (commonsense reasoning), natural language generation (math and code generation), visual understanding (image classification), and visual generation (text-to-image), using a diverse set of model architectures, including LLaMA2 [55], LLaMA3 [20], ViT [16], and Stable Diffusion [48]. We compare StelLA with a diverse set of low-rank adaptation baselines, covering various methodological categories: LoRA [27] as the standard baseline; DoRA [35] as a strong LoRA variant; PiSSA [38] and OLoRA [10] for SVD-based initialization; TriLoRA [19] and MoSLoRA [61] for three-factor decompositions; and ScaledAdamW [66] to represent geometry-aware optimization. Finally, we perform ablation studies to analyze the effect of different components in StelLA.

5.1 Commonsense Reasoning

Models and Datasets. We evaluate the performance of StelLA on the commonsense reasoning benchmark, which assesses the reasoning capabilities of large language models across 8 sub-tasks. Following the setup of Liu et al. [35], we train on the combined data from all sub-tasks and evaluate on the test set. We fine-tune two popular LLM checkpoints, LLaMA2-7B [55] and LLaMA3-8B [20].

Table 1: Accuracy on the commonsense reasoning benchmark. All results are averaged over 3 runs.

Model	Method	Params (%)	BoolQ	PIQA	SIQA	HellaS.	WinoG.	ARC-e	ARC-c	OBQA	Avg.
LLaMA2-7B	LoRA	0.826	72.02	83.46	79.87	90.44	82.69	84.83	71.19	81.53	80.76
	DoRA	0.838	<u>72.67</u>	83.48	79.82	90.82	<u>83.58</u>	85.16	<u>71.27</u>	81.20	<u>81.00</u>
	PiSSA	0.826	71.16	83.89	79.19	91.00	82.87	85.09	69.48	83.93	80.83
	OLoRA	0.826	71.11	82.70	78.64	89.41	81.48	83.58	68.17	80.20	79.41
	TriLoRA	0.828	71.23	80.96	78.33	80.91	77.59	81.76	66.69	79.80	77.16
	MoSLoRA	0.828	71.54	83.84	79.60	90.50	83.19	84.40	69.96	80.47	80.44
	ScaledAdamW	0.826	72.20	83.86	79.67	90.80	82.43	<u>85.55</u>	70.59	81.93	80.88
	StelLA	0.828	73.62	84.87	80.64	91.44	84.50	86.43	72.84	84.33	82.33
LLaMA3-8B	LoRA	0.700	75.16	88.14	80.18	95.41	86.74	90.84	78.70	87.00	85.27
	DoRA	0.710	<u>75.38</u>	88.01	79.94	95.35	86.29	90.54	79.69	86.07	85.16
	PiSSA	0.700	74.67	88.12	<u>80.50</u>	94.98	85.22	90.15	78.87	85.60	84.76
	OLoRA	0.700	74.41	87.68	79.55	94.79	85.40	90.04	78.24	85.00	84.39
	TriLoRA	0.702	73.09	86.64	78.64	93.40	82.88	87.76	75.26	84.30	82.74
	MoSLoRA	0.702	74.88	88.43	80.31	95.50	86.26	90.00	79.86	85.80	85.13
	ScaledAdamW	0.700	75.24	<u>88.57</u>	80.21	<u>95.81</u>	85.11	<u>91.09</u>	<u>80.55</u>	86.60	<u>85.40</u>
	StelLA	0.702	75.91	89.86	81.68	96.41	87.82	91.98	82.34	87.80	86.72

Implementation Details. We compare StelLA against all the aforementioned baselines. For all methods, we insert low-rank adapters into the Q , K , and V projections of the self-attention layers and the up and down projections of the feed-forward layers. For fair comparison, we fix the rank to 32, α to 64, batch size to 16, weight decay to 0, dropout to 0.05, and train for 3 epochs using AdamW. The learning rate is separately tuned for each method and follows a linear decay schedule.

Results. Table 1 shows that StelLA yields consistent, model-agnostic gains on every commonsense sub-task. StelLA lifts the average accuracy of LLaMA2-7B from 81.0% (the best baseline) to 82.3% and that of LLaMA3-8B from 85.4% to 86.7%, corresponding to absolute improvements of about 1.3 points on both models. Crucially, the benefit is uniform: StelLA attains the top score on all eight datasets for both model sizes, showing that its geometry-aware adapters generalize across binary, causal and multiple-choice commonsense formats.

5.2 Math and Code Generation

Models and Datasets. To assess the generative capabilities of StelLA, we conduct experiments on two representative natural language generation (NLG) tasks: mathematical reasoning and code generation. For math-related tasks, the models are fine-tuned on MetaMathQA [65] and evaluated on GSM8K [13] and MATH [25]. For code-related tasks, we train on CodeFeedback [69] and evaluate on HumanEval [11] and MBPP [3].

Implementation Details. We benchmark StelLA against three strong baselines: LoRA [27], DoRA [35], and PiSSA [38]. We follow the experimental protocol of Meng et al. [38]. Each method applies low-rank adaptation to all linear transformations in both self-attention and feed-forward modules. For consistency, we standardize the training configuration across methods: rank is set to 128, α to 128, LoRA dropout and weight decay are both zero, and models are trained with the AdamW optimizer for 1 epoch using a batch size of 128. The learning rate follows a cosine decay schedule and is tuned individually per method to ensure optimal performance.

Results. As summarized in Table 2, StelLA delivers the strongest overall performance (39.30 on average) across the math and code generation benchmarks, surpassing all three established baselines by a comfortable margin (up to +2.69 absolute points on average over DoRA). Importantly, it is the only method that ranks first or second on every benchmark, confirming StelLA’s versatility and effectiveness in challenging natural language generation scenarios.

5.3 Image Classification

Models and Datasets. We assess the performance on image classification tasks using the Vision Transformer [16] pretrained on ImageNet-21K [14]. We fine-tune the Base and Large ViT on 8

Table 2: Results on math and code generation. All results are averaged over 3 runs.

Model	Method	Params (%)	Math		Code		Avg.
			GSM8K	MATH	HumanEval	MBPP	
LLaMA2-7B	LoRA	4.531	64.67	15.33	30.07	41.00	37.76
	DoRA	4.550	64.87	15.32	27.00	39.27	36.61
	PiSSA	4.531	63.73	16.64	31.10	38.20	37.41
	StelLA	4.581	65.43	16.55	33.73	41.47	39.30

Table 3: Comparison of image classification accuracy (%) on 8 datasets averaged over 3 runs.

Model	Method	Params (%)	DTD	EuroSAT	Flowers102	Food101	OxfordPets	Sun397	CIFAR10	CIFAR100	Avg.
ViT Base	LoRA	0.72	<u>79.20</u>	98.98	99.22	90.07	92.94	75.62	<u>98.92</u>	92.60	90.94
	DoRA	0.75	78.56	98.91	<u>99.19</u>	<u>90.15</u>	<u>93.40</u>	<u>75.67</u>	98.96	92.78	<u>90.95</u>
	PiSSA	0.72	77.29	<u>98.93</u>	98.88	89.99	93.21	75.41	<u>98.92</u>	92.28	90.61
	StelLA	0.73	79.89	98.91	99.22	90.17	93.54	76.28	98.88	<u>92.72</u>	91.20
ViT Large	LoRA	0.53	80.11	<u>99.11</u>	99.32	<u>91.30</u>	94.41	77.08	<u>99.23</u>	<u>94.06</u>	91.83
	DoRA	0.55	<u>80.21</u>	99.00	<u>99.28</u>	91.27	94.30	<u>77.19</u>	99.29	94.11	<u>91.83</u>
	PiSSA	0.53	<u>80.21</u>	<u>99.11</u>	99.20	91.10	<u>94.33</u>	77.15	99.11	93.99	91.78
	StelLA	0.54	81.54	99.17	99.19	91.33	94.14	77.51	99.16	93.92	92.00

datasets: Caltech101 [18], CUB200 [57], Cars196 [29], Flowers102 [40], Food101 [8], OxfordPets [42], Sun397 [63], CIFAR10 and CIFAR100 [30], and measure the validation top-1 accuracy.

Implementation Details. We include the following baselines for comparison: LoRA [27], DoRA [35], and PiSSA [38]. We adapt only the query and value matrices of the attention layers in the ViT model. For all methods, we fix the rank and scaling factor α at 16, use a batch size of 128, apply no weight decay, set dropout to 0.1, and train for 10 epochs with the AdamW optimizer and a linear learning-rate schedule. The learning rate itself is tuned independently for each model to ensure a fair comparison.

Results. Table 3 summarizes the results of our experiments. We observe that StelLA delivers the best average performance using both ViT-Base and ViT-Large models, outperforming all other methods across most datasets. Using ViT-Base, StelLA attains the highest accuracy on five of the eight datasets and posts the best overall mean of 91.20%, surpassing the strongest baseline, DoRA, by +0.25 points. Scaling up to ViT-Large, StelLA achieves the best accuracy on four datasets and a mean of 92.00%, outperforming the best baseline, LoRA, by +0.17 points. These results demonstrate that StelLA is a strong contender for image classification tasks.

5.4 Text-to-Image Generation

Models and Datasets. To explore the effectiveness of StelLA in generative vision tasks, we fine-tune text-to-image models on five stylistically diverse datasets sourced from *CivitAI* [12]: *Barbie*, *Cyberpunk*, *Elementfire*, *Expedition*, and *Hornify*. Each dataset includes captions generated using the BLIP model [31]. We conduct experiments on two commonly used latent diffusion [48] based models—Stable Diffusion v1.5 and v2.0, which are equipped with a U-Net architecture composed of ResNet blocks [24].

Implementation Details. We follow standard LoRA fine-tuning recipes in *Diffusers* [56] by injecting LoRA parameters into the cross-attention layers of the U-Net. For benchmarking, we compare StelLA with LoRA, DoRA, and PiSSA and report CLIP score [47] and FID [26], two metrics that respectively measure semantic alignment and visual fidelity. For all methods, we fix the rank and scaling factor α at 4, use a batch size of 8, apply 0.01 weight decay and train for 100 epochs with the AdamW optimizer and a cosine learning-rate schedule. The learning rate itself is tuned independently for each model to ensure a fair comparison.

Table 4: Text-to-Image quantitative results on finetuning SD 1.5 and SD 2.0 to downstream tasks. In most cases, StelLA achieves the best FID and on-par CLIP scores.

Model	Method	Params (%)	BarbieCore		Cyberpunk		ElementFire		Expedition		Hornify	
			FID ↓	CLIP ↑	FID ↓	CLIP ↑	FID ↓	CLIP ↑	FID ↓	CLIP ↑	FID ↓	CLIP ↑
SD 1.5	w/o finetune	-	208.11	30.84	145.37	27.49	253.66	27.78	180.18	27.99	212.90	27.98
	LoRA	0.093	175.48	30.31	127.50	<u>27.62</u>	202.49	<u>27.80</u>	156.34	27.64	180.48	27.24
	DoRA	0.104	<u>175.04</u>	<u>30.36</u>	<u>127.11</u>	27.61	<u>200.77</u>	27.78	<u>155.80</u>	<u>27.65</u>	<u>179.58</u>	<u>27.26</u>
	PiSSA	0.093	299.49	17.32	269.44	16.88	303.18	18.82	291.22	17.55	295.15	17.29
	StelLA	0.093	170.25	30.15	124.46	27.73	194.41	27.83	146.12	27.56	167.53	27.23
SD 2.0	w/o finetune	-	210.45	31.12	158.68	27.42	256.01	27.82	180.74	27.86	214.41	28.15
	LoRA	0.096	171.68	30.72	140.77	<u>27.85</u>	194.53	27.91	159.66	27.53	188.11	<u>27.20</u>
	DoRA	0.107	176.16	<u>30.87</u>	<u>140.71</u>	27.94	197.12	<u>27.92</u>	<u>159.54</u>	<u>27.54</u>	185.40	27.19
	PiSSA	0.096	315.64	16.86	272.60	15.81	284.23	17.96	267.87	16.71	294.59	15.71
	StelLA	0.096	<u>171.83</u>	30.92	135.05	27.83	<u>194.71</u>	28.00	154.06	27.34	177.51	27.05

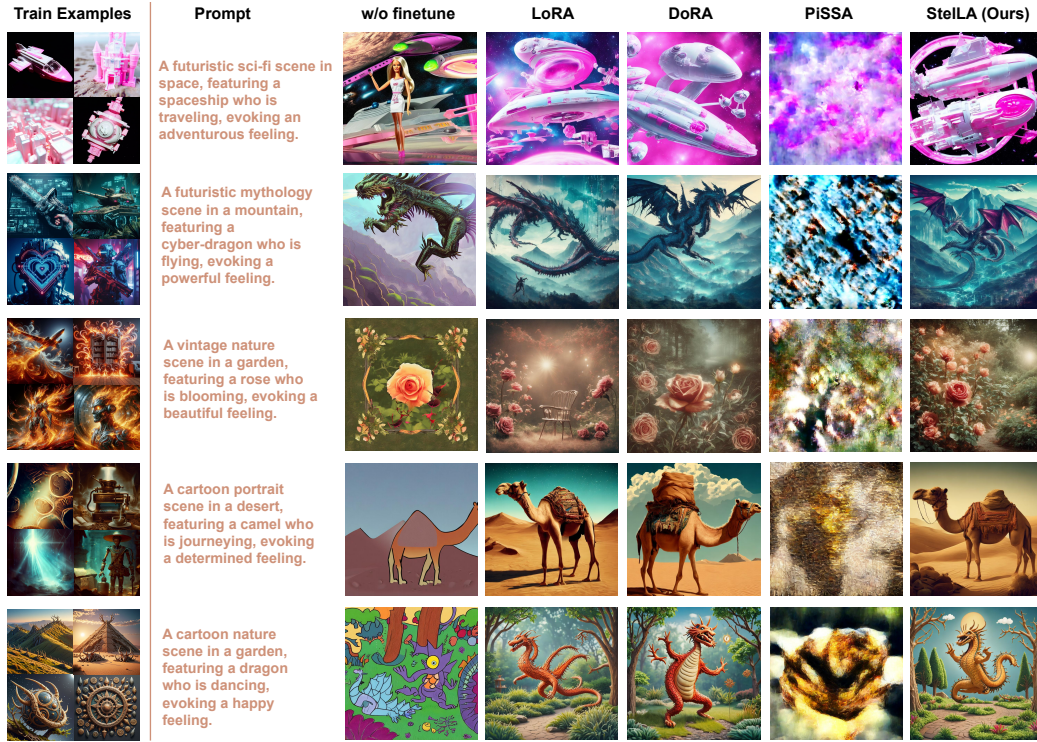


Figure 1: Qualitative results of finetuning SD 1.5 on *BarbieCore*, *Expedition* and *Hornify*.

Results. Quantitative results in Table 4 show that StelLA consistently yields the lowest FID scores, often outperforming the baselines by a notable margin—particularly on datasets like *Expedition* and *Hornify*—while maintaining competitive CLIP scores. In addition, qualitative results shown in Figure 1 reveal that StelLA generates images with strong stylistic consistency (*e.g.*, preserving the original background aesthetics in *BarbieCore*) and high perceptual quality, demonstrating its strength in adapting generative models to downstream domains.

5.5 Ablation Studies

In this section, we analyze the design choices in StelLA. We compare the performance of alternative geometric structures, different initialization strategies, and study the effectiveness of the additional gradient scaling for the Adam optimizer. We also evaluate the effect of parallel transport introduced in Riemannian Adam [6] and the effect of an alternative choice for the retraction operator. The

Table 5: Ablation study on design choices using the Commonsense with LLaMA3-8B. GS and PT denote gradient scaling and parallel transport, respectively. Results are averaged over 3 runs.

Geometry	Initialization	GS	PT	BoolQ	PIQA	SIQA	HellaS.	WinoG.	ARC-e	ARC-c	OBQA	Avg.
Stella	Non-zero	✓	×	75.9	89.9	81.7	96.4	87.8	92.0	82.3	87.8	86.7
Euclidean Quotient	Non-zero	N/A	×	74.0	88.0	80.4	94.9	85.1	89.5	78.1	85.1	84.4
				75.7	88.6	81.0	96.1	86.9	90.9	80.0	86.4	85.7
Stella	Zero	✓	×	75.9	89.2	81.6	96.3	87.4	91.7	81.8	87.9	86.5
	Pseudo-zero			72.8	87.1	80.8	94.8	86.0	89.3	78.2	84.8	84.2
	SVD-major			76.1	89.8	81.5	96.5	87.5	92.3	81.9	88.3	86.7
	SVD-minor			75.9	89.8	81.9	96.4	87.2	92.4	82.0	87.5	86.6
Stella	Non-zero	×	×	76.1	89.7	81.2	96.3	87.4	92.0	80.9	87.6	86.4
Stella	Non-zero	✓	✓	76.2	89.6	81.4	96.4	87.4	91.9	81.7	87.1	86.5

Commonsense reasoning benchmark with the LLaMA3-8B model is used for all ablation studies, and the results are summarized in Table 5.

Geometric Structures. For each weight matrix W to be adapted, there are three learnable matrices U , S , and V in Stella. Since U and V are constrained to lie on the Stiefel manifold, Stella’s geometry is a product of two Stiefel manifolds and a Euclidean manifold, namely, $\text{St}(r, m) \times \mathbb{R}^{r \times r} \times \text{St}(r, n)$. To show the effectiveness of this geometry, we compare it with the following alternatives:

- **Euclidean:** $\mathbb{R}^{m \times r} \times \mathbb{R}^{r \times r} \times \mathbb{R}^{n \times r}$. The simplest geometry is a product of Euclidean spaces, which does not impose any orthonormality constraints on the factors. It is used in previous works such as TriLoRA [19] and MoSLoRA [61].
- **Quotient:** $\text{St}(r, m) \times \mathbb{R}^{r \times r} \times \text{St}(r, n) / (\mathcal{O}(r) \times \mathcal{O}(r))$. The three-factor decomposition of a low-rank matrix $M = USV^\top$ is not unique due to the equivalence relationship $(U, S, V) \sim (UO_1, O_1^\top SO_2, VO_2), \forall O_1, O_2 \in \mathcal{O}(r)$, where $\mathcal{O}(r)$ is the orthogonal group. Factoring out this symmetry, we get a quotient space where each low-rank matrix is uniquely represented. We adapt Stella to use this geometry along with the Riemannian metric defined in [39]. Details of the geometry and its optimization are discussed in Appendix B.

Comparing the performance of these geometries in Table 5, we observe that the product space $\text{St}(r, m) \times \mathbb{R}^{r \times r} \times \text{St}(r, n)$ for Stella consistently outperforms the other two geometries across subtasks. This evidences that (1) The Euclidean three-factor geometry is not as effective as Stella’s geometry, implying that the orthonormality constraints on U and V are beneficial for the low-rank adaptation task, and (2) the Stella’s geometry is more effective than the quotient geometry, which is likely due to the fact that the Riemannian metric [39] on the quotient space was initially designed for the low-rank matrix completion problem rather than low-rank adaptation.

Initializations. We refer to the initialization of Stella introduced in Section 4 as **non-zero** initialization, and compare it with the following initialization strategies:

- **Zero.** Initialize S to be zero, and U and V to be random column-orthonormal matrices.
- **Pseudo-zero.** Same as the non-zero initialization, except that the original weight matrix is modified by subtracting the adapter’s initialization.
- **SVD-major.** Initialize U and V to be the leading r left and right singular vectors of the pretrained weight matrix, respectively. S is initialized as the identity matrix. This setting aligns with the philosophy in PiSSA [38] and can be interpreted as Stella + PiSSA.
- **SVD-minor.** Same as SVD-major, except that the trailing r singular vectors are used. This setting aligns with the philosophy in MiLoRA [58] and can be interpreted as Stella + MiLoRA.

From the results in Table 5, we observe that (1) zero initialization is not as effective as the non-zero initialization. We hypothesize the small value of S creates small gradients for U and V at the beginning of training, leading to slow convergence. (2) Pseudo-zero initialization leads to the worst performance, presumably because it contaminates the pretrained weights with the adapter’s

Table 6: Comparison of the polar retraction with the exponential map on the commonsense reasoning benchmark. Results are averaged over 3 runs.

Geometry	Retraction	BoolQ	PIQA	SIQA	HellaS.	WinoG.	ARC-e	ARC-c	OBQA	Avg.
Stella	Polar	75.91	89.86	81.68	96.41	87.82	91.98	82.34	87.80	86.72
Stella	Exponential Map	75.98	89.52	81.10	96.42	88.27	91.83	82.85	88.13	86.76

initialization. (3) SVD-major and SVD-minor initializations show similar performance as non-zero initialization. This showcases the robustness of our geometric optimization, as it can effectively learn the suitable subspaces regardless of the initialization.

Gradient Scaling (GS) and Parallel Transport (PT). As we have used the AdamW optimizer, we evaluate the effectiveness of the gradient scaling strategy introduced in Section 4 and the parallel transport introduced in Riemannian Adam [6]. Table 5 indicates that our gradient scaling slightly improves the average accuracy from 86.4% to 86.7%, showing its effectiveness in balancing the learning rates of U and V . Moreover, implementing the parallel transport results in the vanilla Riemannian Adam optimization. However, it does not lead to any performance gain compared to our treatment of converting a Euclidean Adam into a Riemannian one in Algorithm 1.

Alternative Choices for Retraction. Other than the polar retraction in Equation (4), the exponential map offers an alternative way of retraction. The exponential map is a locally defined operation that maps a tangent vector at one point to a new point on the manifold by following the geodesic in the given direction for a unit time. Intuitively, it traces the shortest curve on the manifold starting from a point U along the direction of a tangent vector Δ . Formally, let $QR := \Delta - UU^\top \Delta$ be the QR decomposition of $\Delta - UU^\top \Delta$. Then, the exponential map $\exp_U(\Delta)$ can be computed by $\exp_U(\Delta) = UM + QN$, where

$$\begin{pmatrix} M \\ N \end{pmatrix} := \exp \begin{pmatrix} U^\top \Delta & -R^\top \\ R & 0 \end{pmatrix} \begin{pmatrix} I \\ 0 \end{pmatrix}, \quad (7)$$

and $\exp$ is the matrix exponential (not to be confused with the exponential map $\exp_U$).

While the exponential map is geometrically well-founded, it is more expensive to compute than the polar retraction. We compare the performance of polar retraction and the exponential map in Table 6. The two methods achieve nearly identical results (86.72 vs. 86.76). We adopt the polar retraction as the default choice in Stella due to its lower computational cost.

6 Limitations

While Stella demonstrates strong empirical performance across a range of models and tasks, it also comes with several limitations. First, compared to standard LoRA, Stella introduces a small computational overhead due to the three-component formulation, the gradient conversion, and the retraction. Second, we did not explore combining Stella with complementary LoRA variants such as AdaLoRA [68]. These methods introduce orthogonal improvements such as rank scheduling. Finally, due to time and resource constraints, we leave the evaluation of Stella on more model families, such as Mistral or LLaVA, and at very large scales, such as LLaMA2-70B, to future work.

7 Conclusion

We introduced Stella, a subspace-aware extension of Low-Rank Adaptation (LoRA) that explicitly learns input and output subspaces through a three-factor decomposition $USV^\top$, with U and V constrained on Stiefel manifolds. By combining Riemannian optimization with a modular training algorithm that supports arbitrary optimizers, Stella offers a flexible and geometry-aware approach to parameter-efficient fine-tuning. We demonstrated the effectiveness of Stella on a variety of tasks, including language modeling, image classification, and text-to-image generation. Our experiments show that Stella consistently outperforms LoRA and its variants, achieving state-of-the-art results on several benchmarks.

References

- [1] P-A Absil, Robert Mahony, and Rodolphe Sepulchre. Optimization algorithms on matrix manifolds. In *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2009.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [4] Seyedarmin Azizi, Souvik Kundu, and Massoud Pedram. LaMDA: Large model fine-tuning via spectrally decomposed low-dimensional adaptation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9635–9646, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [5] Klaudia Bałazy, Mohammadreza Banaei, Karl Aberer, and Jacek Tabor. Lora-xs: Low-rank adaptation with extremely small number of parameters. *arXiv preprint arXiv:2405.17604*, 2024.
- [6] Gary Becigneul and Octavian-Eugen Ganeva. Riemannian adaptive optimization methods. In *International Conference on Learning Representations*, 2019.
- [7] Massimo Bini, Leander Gırrbach, and Zeynep Akata. Decoupling angles and strength in low-rank adaptation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [8] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision*, pages 446–461. Springer, 2014.
- [9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pages 1877–1901. Curran Associates, Inc., 2020.
- [10] Kerim Büyükakyüz. Olora: Orthonormal low-rank adaptation of large language models. *arXiv preprint arXiv:2406.01775*, 2024.
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [12] Civitai Team. Lora datasets / training data list - civitai dataset guide. <https://civitai.com/articles/2138/lora-datasets-training-data-list-civitai-dataset-guide>, 2025. Accessed: 2025-05-08.
- [13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

- [17] Alan Edelman, Tomás A Arias, and Steven T Smith. The geometry of algorithms with orthogonality constraints. *SIAM journal on Matrix Analysis and Applications*, 20(2):303–353, 1998.
- [18] Li Fei-Fei, Rob Fergus, and Pietro Perona. One-shot learning of object categories. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 594–611. IEEE, 2006.
- [19] Chengcheng Feng, Mu He, Qiuyu Tian, Haojie Yin, Xiaofang Zhao, Hongwei Tang, and Xingqiang Wei. Trilora: Integrating svd for advanced style personalization in text-to-image generation, 2024.
- [20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [21] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [22] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *Transactions on Machine Learning Research*, 2024.
- [23] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. Lora+: Efficient low rank adaptation of large models. In *International Conference on Machine Learning*, pages 17783–17806. PMLR, 2024.
- [24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [25] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- [26] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017.
- [27] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [28] Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with lora. *arXiv preprint arXiv:2312.03732*, 2023.
- [29] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pages 554–561, 2013.
- [30] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [31] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International conference on machine learning*, pages 12888–12900. PMLR, 2022.
- [32] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online, 2021. Association for Computational Linguistics.
- [33] Vladislav Lialin, Sherin Muckatira, Namrata Shivagunde, and Anna Rumshisky. ReLoRA: High-rank training through low-rank updates. In *The Twelfth International Conference on Learning Representations*, 2024.
- [34] Kai Lion, Liang Zhang, Bingcong Li, and Niao He. PoLAR: Polar-decomposed low-rank adapter representation. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025.
- [35] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Forty-first International Conference on Machine Learning*, 2024.
- [36] Yun Long, Haifeng Luo, and Yu Zhang. Evaluating large language models in analysing classroom dialogue. *npj Science of Learning*, 9(1):60, 2024.

- [37] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- [38] Fanxu Meng, Zhaohui Wang, and Muhan Zhang. PiSSA: Principal singular values and singular vectors adaptation of large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [39] Bamdev Mishra and Rodolphe Sepulchre. R3mc: A riemannian three-factor algorithm for low-rank matrix completion. In *53rd IEEE Conference on Decision and Control*, pages 1137–1142. IEEE, 2014.
- [40] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, 2008.
- [41] Fabian Paischer, Lukas Hauenberger, Thomas Schmied, Benedikt Alkin, Marc Peter Deisenroth, and Sepp Hochreiter. One initialization to rule them all: Fine-tuning via explained variance adaptation. In *NeurIPS 2024 Workshop on Fine-Tuning in Modern Machine Learning: Principles and Scalability*, 2024.
- [42] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3498–3505, 2012.
- [43] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [44] PyTorch. torch.linalg.svd — pytorch 2.7 documentation. <https://docs.pytorch.org/docs/2.7/generated/torch.linalg.svd.html#torch-linalg-svd>, 2025. Accessed: 2025-05-10.
- [45] Yujia Qin, Xiaozhi Wang, Yusheng Su, Yankai Lin, Ning Ding, Jing Yi, Weize Chen, Zhiyuan Liu, Juanzi Li, Lei Hou, Peng Li, Maosong Sun, and Jie Zhou. Exploring universal intrinsic task subspace for few-shot learning via prompt tuning. *IEEE/ACM Trans. Audio, Speech and Lang. Proc.*, 32:3631–3643, 2024.
- [46] Zeju Qiu, Weiyang Liu, Haiwen Feng, Yuxuan Xue, Yao Feng, Zhen Liu, Dan Zhang, Adrian Weller, and Bernhard Schölkopf. Controlling text-to-image diffusion by orthogonal finetuning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [47] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [48] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- [49] Soumava Kumar Roy, Zakaria Mhammedi, and Mehrtash Harandi. Geometry aware constrained optimization techniques for deep learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4460–4469, 2018.
- [50] Dayana Savostianova, Emanuele Zangrando, Gianluca Ceruti, and Francesco Tudisco. Robust low-rank training via approximate orthonormal constraints. *Advances in Neural Information Processing Systems*, 36: 66064–66083, 2023.
- [51] Andrew M Saxe, James L McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv preprint arXiv:1312.6120*, 2013.
- [52] Steffen Schotthöfer, H Lexie Yang, and Stefan Schnake. Dynamical low-rank compression of neural networks with robustness under adversarial attacks. *arXiv preprint arXiv:2505.08022*, 2025.
- [53] Steffen Schotthöfer, Emanuele Zangrando, Gianluca Ceruti, Francesco Tudisco, and Jonas Kusch. GeoloRA: Geometric integration for parameter efficient fine-tuning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [54] Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Mohamed Amin, Le Hou, Kevin Clark, Stephen R Pfohl, Heather Cole-Lewis, et al. Toward expert-level medical question answering with large language models. *Nature Medicine*, pages 1–8, 2025.
- [55] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

- [56] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, Yiyi Xu, Steven Liu, and Thomas Wolf. Diffusers: State-of-the-art diffusion models. <https://github.com/huggingface/diffusers>, 2022.
- [57] Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona, and Serge Belongie. The caltech-ucsd birds-200-2011 dataset. In *California Institute of Technology*, 2011.
- [58] Hanqing Wang, Yixia Li, Shuo Wang, Guanhua Chen, and Yun Chen. MiLoRA: Harnessing minor singular components for parameter-efficient LLM finetuning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4823–4836, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.
- [59] Shaowen Wang, Linxi Yu, and Jian Li. LoRA-GA: Low-rank adaptation with gradient approximation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [60] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- [61] Taiqiang Wu, Jiahao Wang, Zhe Zhao, and Ngai Wong. Mixture-of-subspaces in low-rank adaptation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7880–7899, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [62] Yecheng Wu, Zhuoyang Zhang, Junyu Chen, Haotian Tang, Dacheng Li, Yunhao Fang, Ligeng Zhu, Enze Xie, Hongxu Yin, Li Yi, Song Han, and Yao Lu. VILA-u: a unified foundation model integrating visual understanding and generation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [63] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3485–3492, 2010.
- [64] Greg Yang and Etai Littwin. Tensor programs ivb: Adaptive optimization in the infinite-width limit. *arXiv preprint arXiv:2308.01814*, 2023.
- [65] Longhui Yu, Weisen Jiang, Han Shi, Jincheng YU, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [66] Fangzhao Zhang and Mert Pilanci. Riemannian preconditioned lora for fine-tuning foundation models. In *International Conference on Machine Learning*, pages 59641–59669. PMLR, 2024.
- [67] Fangzhao Zhang and Mert Pilanci. Spectral adapter: Fine-tuning in spectral space. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [68] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations*, 2023.
- [69] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12834–12859, Bangkok, Thailand, 2024. Association for Computational Linguistics.
- [70] Weiming Zhuang, Chen Chen, Zhizhong Li, Sina Sajadmanesh, Jingtao Li, Jiabo Huang, Vikash Sehwal, Vivek Sharma, Hirotaka Shinozaki, Felan Carlo Garcia, et al. Argus: A compact and versatile foundation model for vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

A Intuitive Example about Stiefel Manifold

To help readers understand the Stiefel manifold, we provide an intuitive example in low dimensions. In Section 3, we provided general equations for various geometric concepts. Here, we apply those ingredients to the following specific example.

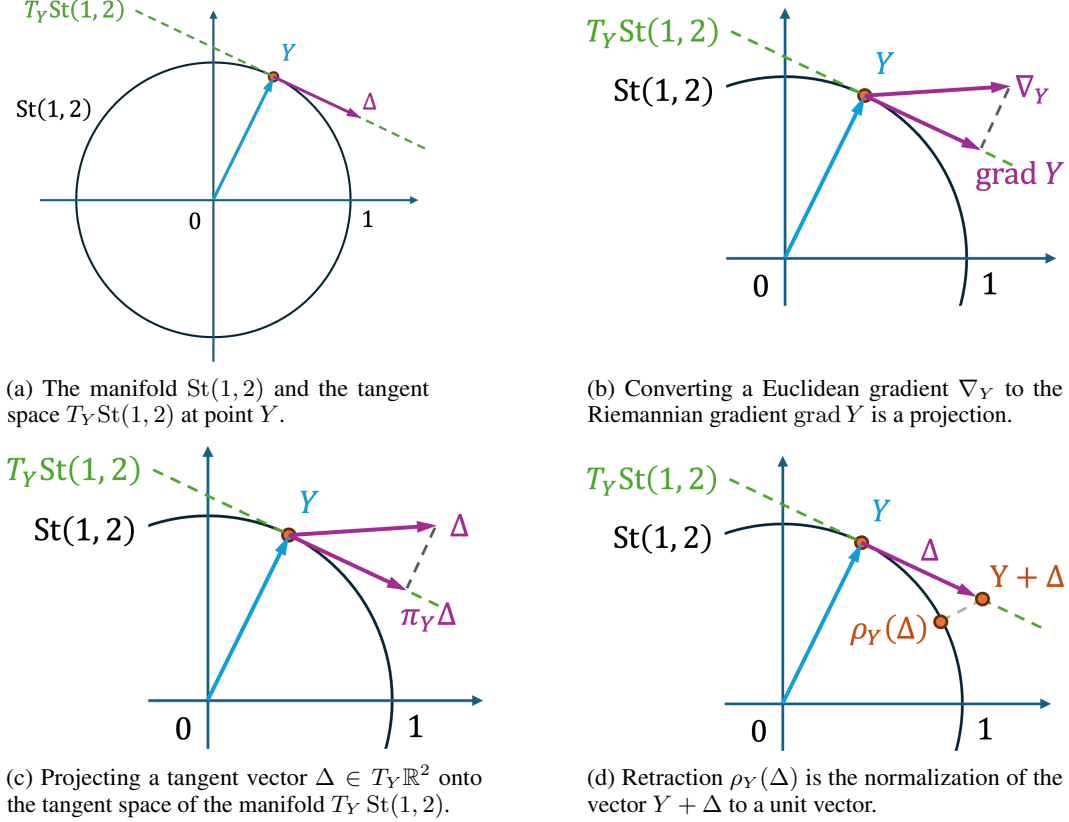


Figure 2: An example Stiefel manifold $\text{St}(1, 2)$: the unit circle on the two-dimensional plane.

Manifold. Consider $\text{St}(1, 2) = \{Y = [y_1, y_2]^\top \in \mathbb{R}^{2 \times 1} | Y^\top Y = y_1^2 + y_2^2 = 1\}$, the Stiefel manifold of orthonormal 1-frames in $\mathbb{R}^2$, which consists of all unit vectors in the plane, *i.e.*, the unit circle. See Figure 2a for an illustration.

Tangent Space. For a point $Y = [y_1, y_2]^\top \in \text{St}(1, 2)$, a tangent vector $\Delta = [\delta_1, \delta_2]^\top$ must satisfy the constraint $Y^\top \Delta + \Delta^\top Y = 0$, which leads to the condition $y_1 \delta_1 + y_2 \delta_2 = 0$, namely $Y \perp \Delta$. The tangent space at Y is the set of all vectors Δ that are orthogonal to Y . See Figure 2a for an illustration.

Riemannian Metric. For two tangent vectors Δ_1, Δ_2 at a point Y , the canonical metric defines the inner product as

$$g_Y(\Delta_1, \Delta_2) = \text{tr}(\Delta_1^\top (I_n - \frac{1}{2} Y Y^\top) \Delta_2) = \text{tr}(\Delta_1^\top \Delta_2) - \frac{1}{2} \text{tr}(\Delta_1^\top Y Y^\top \Delta_2) = \Delta_1^\top \Delta_2, \quad (8)$$

which equals to the standard inner product in $\mathbb{R}^2$. The last equation holds because $Y^\top \Delta_2 = 0$ and $\Delta_1^\top \Delta_2$ is a scalar.

Riemannian Gradient. Suppose the Euclidean gradient of a function f at Y is $\nabla f(Y)$, which is abbreviated as ∇_Y . Then the Riemannian gradient $\text{grad } f(Y)$, abbreviated as $\text{grad } Y$, is given by

$$\text{grad } Y = \nabla_Y - Y \nabla_Y^\top Y = \nabla_Y - (\nabla_Y^\top Y) Y, \quad (9)$$

which is the projection of the Euclidean gradient onto the tangent space at Y . The second term $(\nabla_Y^\top Y)Y$ is the component of the Euclidean gradient that is not tangent to the manifold. See Figure 2b for an illustration.

Tangent Space Projection. The projection of a vector Δ onto the tangent space at Y is given by

$$\pi_Y(\Delta) = \Delta - Y \text{symm}(\Delta^\top Y) = \Delta - (\Delta^\top Y)Y, \quad (10)$$

which is the component of Δ that is tangent to the manifold. The second term $(\Delta^\top Y)Y$ is the component of Δ that is not tangent to the manifold. See Figure 2c for an illustration.

Retraction. For a tangent vector Δ at Y , the retraction is given by

$$\rho_Y(\Delta) = \text{uf}(Y + \Delta) = \frac{Y + \Delta}{\|Y + \Delta\|}, \quad (11)$$

which in the $\text{St}(1, 2)$ case is equivalent to normalizing the vector $Y + \Delta$ to a unit vector. See Figure 2d for an illustration.

B Details of the Quotient Geometry

The three-factor decomposition of a low-rank matrix $M = USV^\top$ is not unique due to the equivalence relationship $(U, S, V) \sim (UO_1, O_1^\top SO_2, VO_2), \forall O_1, O_2 \in \mathcal{O}(r)$, where $\mathcal{O}(r)$ is the orthogonal group. Each rank- r matrix has infinite representations. For example,

$$(UO_1)(O_1^\top SO_2)(VO_2)^\top = U(O_1O_1^\top)S(O_2O_2^\top)V^\top = USV^\top. \quad (12)$$

Factoring out this symmetry, we get a quotient space $\text{St}(r, m) \times \mathbb{R}^{r \times r} \times \text{St}(r, n) / (\mathcal{O}(r) \times \mathcal{O}(r))$, where each low-rank matrix is uniquely represented.

Mishra and Sepulchre [39] proposed a Riemannian metric for this quotient space. We adapt StellA to use this Riemannian metric and refer to this setting as the Quotient geometry. The Riemannian metric in [39] is induced by the block approximation of the Hessian of $\|USV^\top - W\|_F^2$. For two tangent vectors $(\xi_U, \xi_S, \xi_V), (\eta_U, \eta_S, \eta_V)$ at a point (U, S, V) , the Riemannian metric is defined as

$$g_{(U,S,V)}((\xi_U, \xi_S, \xi_V), (\eta_U, \eta_S, \eta_V)) = \text{tr}(SS^\top \xi_U^\top \eta_U) + \text{tr} \xi_R^\top \eta_R + \text{tr}(S^\top S \xi_V^\top \eta_V), \quad (13)$$

where $SS^\top$ and $S^\top S$ act as preconditioners which improve the convergence in the low-rank matrix completion problem. Details of the geometric optimization in this space is discussed in [39], including the equations for the Euclidean gradient to Riemannian gradient conversion, the tangent vector projection $\pi_{(U,S,V)}$, and the retraction $\rho_{(U,S,V)}$. Note that Mishra and Sepulchre [39] used the conjugate-gradient optimization algorithm to solve the low-rank matrix completion problem. However, with the operators such as $\pi_{(U,S,V)}$ and $\rho_{(U,S,V)}$ defined, it is straightforward to adapt to the Riemannian Adam algorithm following Algorithm 1. Specifically, we use [39, Eq. (9)] to convert the Euclidean gradient to the Riemannian gradient, [39, Eq. (5)] for the projection onto tangent space, and [39, Eq. (7)] for the retraction.

C Details of Hyperparameters

We provide the detailed hyperparameters used in our experiments to ensure full reproducibility. For each benchmark, all compared methods share a common set of hyperparameters—such as rank, batch size, weight decay, and training schedule—which are outlined in Section 5. The only exception is the learning rate, which we individually tune for each method to ensure fair comparison. In this section, we list the specific learning rates used for each algorithm across benchmarks.

Commonsense Reasoning. We explore the learning rate in $\{0.0005, 0.0001, 0.00005, 0.00001\}$ for all algorithms, with the chosen values shown in Table 7.

Math and Code Generation. The search space for the learning rates is $\{0.0005, 0.0002, 0.00002\}$ for all methods. We list the selected learning rates in Table 8.

Table 7: Learning Rates for Commonsense Reasoning Experiments.

Model	Method	Learning Rate
LLaMA2-7B	LoRA	0.0001
	DoRA	0.0001
	PiSSA	0.00005
	OLoRA	0.00005
	TriLoRA	0.00001
	MoSLoRA	0.0001
	ScaledAdamW	0.0001
	StelLA	0.0005
LLaMA3-8B	LoRA	0.0001
	DoRA	0.0001
	PiSSA	0.00005
	OLoRA	0.00005
	TriLoRA	0.0001
	MoSLoRA	0.0001
	ScaledAdamW	0.00005
	StelLA	0.0005

Table 8: Learning Rates for Math and Code Generation Experiments.

Model	Method	Math	Code
LLaMA2-7B	LoRA	0.0002	0.0002
	DoRA	0.0002	0.0002
	PiSSA	0.0002	0.0002
	StelLA	0.0005	0.0005

Image Classification. The learning rate is tuned in $\{0.0001, 0.0005, 0.001, 0.005\}$ for all methods. The chosen learning rates are summarized in Table 9.

Text to Image Generation. We search the learning rate over $\{0.0001, 0.0002, 0.0004, 0.0008\}$ for all methods, with the selected values reported in Table 10. For each method, we use the same learning rate for all five CivitAI datasets.

D Details of the Computational Cost for Algorithm 1

In Algorithm 1, for each of the Stiefel parameter (*i.e.*, U and V), it has the following operations:

1. Converting the Euclidean gradient to the Riemannian gradient using Equation (2).
2. Projecting the update direction onto the tangent space using Equation (3).
3. Retracting the updated point back onto the Stiefel manifold using Equation (4).

Among these, the first two steps involve only basic matrix multiplications and additions, and thus incur negligible computational overhead. The dominant cost arises from the retraction step, which requires a polar decomposition of the matrix $Y + \Delta$. This polar decomposition is typically computed via singular value decomposition (SVD) as follows.

Suppose the SVD of $(Y + \Delta) \in \mathbb{R}^{m \times r}$ with $m \gg r$ is expressed as $Y + \Delta = U\Sigma V^\top$, where $U \in \mathbb{R}^{m \times r}$ and $V \in \mathbb{R}^{r \times r}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{r \times r}$ is a diagonal matrix. Then the orthonormal factor in the polar decomposition is given by

$$\text{uf}(Y + \Delta) = UV^\top. \quad (14)$$

Thus, the retraction step has a cost dominated by computing the SVD of an $m \times r$ rectangular matrix.

Theoretically, the computational complexity of the polar retraction step is $O(mr^2)$ for tall matrices ($m \gg r$) due to the SVD computation. But practically, there are efficient SVD algorithms for tall matrices that can reduce the computational cost significantly. Specifically, we use the `gesvda`

Table 9: Learning Rates for Image Classification Experiments.

Model	Method	DTD	EuroSAT	Flowers102	Food101	OxfordPets	Sun397	CIFAR10	CIFAR100
ViT Base	LoRA	0.005	0.005	0.005	0.001	0.005	0.001	0.001	0.001
	DoRA	0.005	0.005	0.005	0.001	0.005	0.001	0.001	0.001
	PiSSA	0.005	0.005	0.005	0.001	0.001	0.001	0.0005	0.001
	StelLA	0.005	0.005	0.005	0.001	0.005	0.001	0.001	0.001
ViT Large	LoRA	0.001	0.005	0.001	0.0005	0.001	0.0005	0.001	0.001
	DoRA	0.001	0.005	0.001	0.0005	0.0005	0.0005	0.001	0.0005
	PiSSA	0.001	0.001	0.001	0.0005	0.0005	0.0005	0.0001	0.0005
	StelLA	0.005	0.005	0.001	0.001	0.001	0.001	0.001	0.0005

Table 10: Learning Rates for Text to Image Generation Experiments.

Model	Method	Learning Rate (same for all 5 tasks)
SD 1.5 & SD 2.0	LoRA	0.0001
	DoRA	0.0001
	PiSSA	0.0001
	StelLA	0.0008

solver [44], which is a CUDA-accelerated SVD implementation that can handle tall matrices efficiently. Below is a micro-benchmark of different SVD solvers for the matrix shapes used in StelLA for the commonsense benchmark on LLaMA3-8B. Results in Table 11 show that the gesvda solver is significantly faster than the default GPU solver in PyTorch.

The speed could be further improved by using batch processing to increase parallelism. Specifically, we can stack all the low-rank matrices U and V with the same shape into a batch, and then perform the polar retraction step on the batch of matrices. Table 12 is a micro-benchmark of the SVD using batch processing on the matrix shapes used in StelLA for the commonsense benchmark on LLaMA3-8B (there are 192, 64, and 64 matrices with shapes 4096×32 , 1024×32 , and 14336×32 , respectively). The results show that the batched SVD is significantly faster than the sequential one, and the polar retraction step is no longer the bottleneck of the training time.

In practice, training a LoRA-adapted LLaMA3-8B model on a commonsense reasoning benchmark takes approximately 4.5 hours on a single H100 GPU, whereas training the same model with StelLA takes around 5.2 hours, about only 15% slower than vanilla LoRA.

E Discussion on Scale Stability

The scale stability of LoRA refers to the property that neither the activations nor the gradients explode or vanish as the rank r , input dimension n , and output dimension m grow infinity [28, 59]. More precisely, *forward stability* is achieved if, assuming the input to the adapter has i.i.d. entries with second moment $\Theta_{r,m,n}(1)$, the output of the adapter also maintains a second moment of the same order. Similarly, *backward stability* is achieved if, when the gradient of the loss w.r.t. the adapter output has second moment $\Theta_{r,m,n}(1)$, the gradient w.r.t. the adapter input also remains at $\Theta_{r,m,n}(1)$. Letting γ denote the scaling coefficient applied to the low-rank update, rsLoRA [28] shows that the original LoRA choice of $\gamma = \frac{\alpha}{r} = \Theta(\frac{1}{r})$ is not rank stable. Instead, they propose that γ should scale as $\Theta(1/\sqrt{r})$ to maintain stability.

In StelLA, we initialize U and V as random orthogonal matrices and S to the identity matrix. This setup satisfies the main assumptions in Theorem 3.2 of [59], which analyzes the scale stability of LoRA. Following their analysis, we assess the scale stability of StelLA at initialization as follows.

Let the adapter compute $y = \gamma USV^\top x$ for an input vector x , where γ is the scaling factor. Since S is initialized to be identity, this simplifies to $y = \gamma UV^\top x$. The forward pass can be expressed component-wise as

$$y_i = \gamma \sum_{j=1}^r \sum_{k=1}^n U_{ij} V_{kj} x_k, \quad 1 \leq i \leq m. \quad (\text{Forward}) \quad (15)$$

Table 11: Runtime of different SVD solvers in PyTorch on H100 GPU.

Matrix Shape	Default Solver (ms)	GESVDA Solver (ms)	Speedup of GESVDA
4096×32	0.938	0.641	1.46×
1024×32	0.795	0.632	1.26×
14336×32	1.530	0.654	2.34×

Table 12: Runtime of SVD with/without batch processing on H100 GPU. The GESVDA solver is used for both batched and sequential processing.

Matrix Shape	Batch Size	Batched (ms)	Sequential (ms)	Speedup of Batched
192×4096×32	192	5.80	123.1	22.22×
64×1024×32	64	1.62	40.4	24.94×
64×14336×32	64	2.89	41.8	14.46×

To analyze the scale, we compute the second moment:

$$\mathbb{E}[y_i^2] = \gamma^2 \sum_{j_1=1}^r \sum_{j_2=1}^r \sum_{k_1=1}^n \sum_{k_2=1}^n \mathbb{E}[U_{ij_1} V_{k_1 j_1} x_{k_1} U_{ij_2} V_{k_2 j_2} x_{k_2}] \quad (16)$$

$$= \gamma^2 \sum_{j_1=1}^r \sum_{j_2=1}^r \sum_{k_1=1}^n \sum_{k_2=1}^n \mathbb{E}[U_{ij_1} U_{ij_2}] \mathbb{E}[V_{k_1 j_1} V_{k_2 j_2}] \mathbb{E}[x_{k_1} x_{k_2}] \quad (17)$$

$$= \gamma^2 \sum_{j_1=1}^r \sum_{j_2=1}^r \sum_{k=1}^n \mathbb{E}[U_{ij_1} U_{ij_2}] \mathbb{E}[V_{k j_1} V_{k j_2}] \mathbb{E}[x_k^2] \quad (18)$$

$$= \gamma^2 \sum_{j=1}^r \sum_{k=1}^n \mathbb{E}[U_{ij}^2] \mathbb{E}[V_{kj}^2] \mathbb{E}[x_k^2] \quad (19)$$

$$= \gamma^2 r n \frac{1}{m} \frac{1}{n} \Theta_{r,m,n}(1) = \frac{\gamma^2 r}{m} \Theta_{r,m,n}(1). \quad (20)$$

Hence, the forward pass remains scale-stable in the beginning of the training if $\gamma = \Theta(\sqrt{\frac{m}{r}})$. Equation (18) is derived from the independence of x_{k_1} and x_{k_2} , where $\mathbb{E}[x_{k_1} x_{k_2}] = 0$ when $k_1 \neq k_2$. Equation (19) and Equation (20) are derived from the fact that for a random unit vector $a \in \mathbb{R}^n$, $\mathbb{E}[a_i a_j] = 0$ when $i \neq j$ and $\mathbb{E}[a_i^2] = \frac{1}{n}$.

For the backward pass, the gradient with respect to the input is given by $g = \frac{\partial \mathcal{L}}{\partial x} = \gamma V U^\top v$, where $v = \partial \mathcal{L} / \partial y$. Then,

$$g_i = \gamma \sum_{j=1}^r \sum_{k=1}^m V_{ij} U_{kj} v_k, \quad 1 \leq i \leq n. \quad (\text{Backward}) \quad (21)$$

and the second moment becomes:

$$\mathbb{E}[g_i^2] = \gamma^2 \sum_{j_1=1}^r \sum_{j_2=1}^r \sum_{k_1=1}^m \sum_{k_2=1}^m V_{ij_1} U_{k_1 j_1} v_{k_1} V_{ij_2} U_{k_2 j_2} v_{k_2} \quad (22)$$

$$= \gamma^2 \sum_{j=1}^r \sum_{k=1}^m \mathbb{E}[v_{ij}^2] \mathbb{E}[U_{kj}^2] \mathbb{E}[v_k^2]. \quad (23)$$

$$= \gamma^2 r m \frac{1}{n} \frac{1}{m} \Theta_{r,m,n}(1) = \frac{\gamma^2 r}{n} \Theta_{r,m,n}(1). \quad (24)$$

which implies the backward pass is stable in the beginning of the training if $\gamma = \Theta(\sqrt{\frac{n}{r}})$.

In practice, we adopt the coefficient $\gamma = \frac{\alpha}{r}$ to be consistent with LoRA. While this choice does not guarantee theoretical scale stability, it does not negatively affect training in practice, as α can be adjusted empirically. Since m , n , and r are fixed for a given model, tuning α allows us to ensure stable and convergent training across tasks.